M1 MOSIG 2025
OS LAB 2 by BUDOVICH Ivan & BEZRUKOV Andrei

The platform on which we have tested our code is:
Our personal PCs are using Virtual machines on Windows (VirtualBox - Andrei, VMware Workstation - Ivan).
· the Linux distribution - Ubuntu 24.04;
· the CPU type - Intel x86_64

Implemented features and design choices:

Allocation policies (basic and advanced versions):

- First Fit (Tests 1-6 passed)
  When allocating a block, we're checking by the condition if there's enough free space to split the free zone to get a new free block from the remainder or not. We need to ensure that the new free block has enough space for a larger free header, so we're using the maximum size of free and allocated headers. Finally, when the block is allocated, we return the pointer to the payload by casting to (char *) in order to count the address in bytes.

- Best Fit (Tests 1-6 passed)
  Here, we add two additional pointers: best_free is the best-fitting block (with the least remainder), best_prev_free is the previous block of best_free (used for linking). So when removing or splitting the block, we don't need an additional traversal. We also store best_diff, which is the difference between the current free block size and the requested size. Then we scan the free list to find the best option, and if we find it, similarly to FF, we check if this block is big enough to be split and return the pointer to the payload. This approach is efficient in terms of space complexity and reduces fragmentation.

- Next Fit (Tests 1-6 passed)
  We add a global pointer p_next_fit - it remembers where the allocation stopped, starts from there, and if we didn't find the block after reaching the end of the memory pool, we return to the start. The variable is_loop_passed is used to check whether we wrapped around, so in the condition, we use it to make sure we traverse the list fully only once (so there won't be infinite loops). Afterwards, similar to previous techniques, we try to split the block, update the p_next_fit (moving it to the next free block or to the start), and return the payload. In general, it's more efficient in terms of speed.

Freeing memory (basic and advanced versions): our free list is a singly linked list sorted by increasing address, where each free block header stores size and a pointer to the next free block. We're basically going through the list and comparing addresses until we find the needed block and insert it into the list at the right position. Then, we check if the next free block is adjacent to the block we just freed, and merge them in that case, we go through the same check with the previous free block.

Printing memory state (basic and advanced versions): we're displaying allocated bytes as "+" and free bytes as "-" simply for readability reasons. "-" don't coalesce together (like underscores "*"),* so it's quite easy to count free bytes. We represented memory zones including 64 bytes in one row because it's a power of 2 and it can be conveniently shown in the terminal *(print_mem_state()*), so taking into account the full MEM_POOL_SIZE (which is 1024 bytes by default), we'll get rows with the same length. Also, it's not advisable generally to make one line of text or symbols longer than 70-80 characters in order to find the start of the next line simply, so we're following this rule as well. Regarding implementation, it might be slow due to the nested loops O(N^2); we're going through every byte of the memory pool instead of going through the list of free blocks, for example, but it's acceptable for debugging reasons.

Alignment (advanced version):
As we assume that the mem poop size is aligned, we only need to align the headers and the content size (separately).
To do it we can use the following mathematical operation:
ALIGNED = (NON_ALIGNED + MEM_ALIGNMENT) / MEM_ALIGNMENT * MEM_ALIGNMENT
It will allow us to truncate the size up to the next number which can be divided by the mem alignment without reminder.
As this number is a power of two we can use the following binary operation:
ALIGNED = (NON_ALIGNED + MEM_ALIGNMENT - 1) & NOT(MEM_ALIGNMENT - 1)

Safety checks (advanced version):

Returning a valid pointer when the allocation size is 0
When the size of the requested block is zero, we look for an appropriate block which is not less (not including header) than the maximum of the headers minus the minimum of the headers (in our case the allocated header is less but the next safety check could change it).

Corrupting the allocator metadata:
The simplest check we figured out is to add some magic number into the header of the allocated block. We selected 0x5A5A5A5A which would be a complex repeated combination of bits. The downside is that the eldest part (in case of 64-bit system) will be zero.
Upon the allocation, we assign the magic number to the block.
When we free the block, we check if this field has this value. Unless, we throw an error as the further processing can corrupt the whole memory pool. Otherwise, we 'zeroify' and go on the freeing.

M1 MOSIG 2025
OS LAB 2 by BUDOVICH Ivan & BEZRUKOV Andrei

Forgetting to free memory:

We implemented the method where starting with the heap head we check if there are gaps in the list of the free blocks.
First we check if there are any free blocks, otherwise the whole pool is leaked.
Then we check if there is a gap between the heap head and the first free block.
After it, in the loop, we iterate through the free blocks and check if there is space between them. (If there are no leaks, we have only one block, so we skip it)
Finally, we check if there is a gap between last block end and the end of the heap.

Calling free() incorrectly:

We implemented as a part of the free method the check that the point is not NULL and is inside the boundaries of the mem_pool. We decided just to ignore such calls in the base version but exit in the .

Answers to the questions:

General design principles

1. Metadata are the data that describe the memory blocks inside your memory zone. Where are the metadata stored? *At the beginning of the block, in the header, memblk_free_t or memblk_allocated_t.*
2. Do you have to keep a list for allocated blocks? *No, it's enough to store only free blocks (in a singly linked free list).*
3. When a block is allocated, which address should be returned to the user? *We return p_payload, which is a pointer to the start of the payload (just after the header)*
4. When a block is freed by the user, which address is used as an argument to the function free? *Similarly, we use the pointer to the start of the payload.*
5. What should be done when reintegrating a block in the list of free blocks? *We need to sort the list of free blocks (based on the position), to merge free blocks if they're adjacent - after the merge, update pointers from the list of free blocks, to add freed blocks to the list.*
6. When a block is allocated inside a free memory zone, one must take care of how the memory is partitioned. The zone might be bigger than the amount of memory requested. What should we do with the remaining memory space? *We should split it and add the new remaining space as a block to the free list.*
7. Following the previous question, could there be an issue in the case that the remaining memory space is very small? What should we do in this case? *Yes, in that case, we allocate more memory with the remaining size included. That case takes place if the size of the remaining space is smaller than the maximum size of headers memblk_free_t or memblk_allocated_t (in our case, it's memblk_free_t, though we still use a macro).*
8. What is the minimum possible size for a free block? And for an allocated block? *For a free block, it's the size of header memblk_free_t, for an allocated block, it's the size of header memblk_allocated_t. It happens when we try to allocate 0 bytes.*

Basic allocator implementation

M1 MOSIG 2025
OS LAB 2 by BUDOVICH Ivan & BEZRUKOV Andrei

Question 1. For the free list of your allocator, different designs are possible: using a singly linked list can be sufficient but it is also possible to choose a doubly linked list. In any case, describe one advantage and one drawback for the choice that you have made.

Answer. We initially planned to start with a single linked list and then switch if it makes sense. However, we decided to keep the simple approach with the linked list as it works for the most cases.
Advantage: it consumes less memory and as far as in both cases for all three techniques the allocation complexity is O(n) in the worst case it is efficient as well.
Disadvantage: For freeing memory, though, the double linked list can speed up the search and simplify the merging of the adjacent blocks.

Question 2. For this lab, the linked list of free blocks must be sorted by increasing block addresses. Describe one advantage and one drawback of this approach.
Answer. As proposed, we use a sorted list.
Advantage: Sorted lists decrease the segmentation of the blocks as far as we always merge the free adjacent blocks. (It is possible but not rational to merge in non-sorted list)
Disadvantage: Insertion complexity (in the worst case) for freeing is O(N), instead of O(1) for non-sorted (if we assume that we just push/enqueue the block).

Additional tests scenario
As far as tests do not provide scenarios for allocation of zero bytes (and because we initially did not notice it), we implemented the version where allocation zero returns NULL. We reread the requirements, we commented on several lines of code where there are checks for zero, and ran the test where we allocated the first pointer as zero and then we printed a state of memory (a0 p). It resulted in SEG FAULT. After debugging with logs we discovered that we overwrite the pointer for the next block with the size of the new separated block (because allocated header is less than free header). It had led us to initially figure out the assignment order but later we understood that we should use the minimum size of the block equal to the difference between max of the headers and min of the headers.
Later, after the fix, we retested it with: a0 a10 f2 p f1 p